

Deep Learning Models in Robotics: Literature Review and Experimentation

Mohamed Arham Ishak

University of Manchester

27 March 2025

1 Literature Review: Autonomous Navigation of Robots Using Deep Learning Techniques

1.1 Introduction

The autonomous navigation of robots is a field that has undergone extensive research in recent times in order to overcome the challenges of navigating the real world – as seen with underwater robot navigation due to attenuation of undersea light with depth, making it difficult for conventional visual sensors to capture usable images [1]. Traditional data fusion techniques used in Multi-Sensor Integrated Navigation/Positioning Systems (MSINPS) such as the Kalman Filter (KF) for state estimation struggle to effectively process the wealth of sensor data, especially in dynamic environments [4], which has resulted in the need for deep learning in autonomous robot navigation.

This review will briefly discuss three techniques commonly used for deep learning in this domain: Convolutional Neural Networks (CNNs), Deep Reinforcement Learning (DRL), and Optimised VSLAM.

1.2 Convolutional Neural Networks (CNNs)

Convolutional Neural Networks are a type of Neural Network optimised for processing pictures and other grid-like input. They are highly competent in feature extraction and pattern identification. Transfer learning can be applied by training CNNs on big datasets for general navigation in a simulated, controlled environment and then fine-tuning them for specialised environments like underwater navigation. According to performance results from Priscilla Sasi et al., 2023 [1], their proposed CNN-based method with 91% accuracy outperforms algorithms such as DRL (79%), FASTSLAM (82%) and the Extended Kalman Filter (71%).

1.3 Optimised Visual SLAM

Optimised Visual SLAM aims to enhance traditional SLAM by combining RGB-D cameras with the ORB SLAM2 framework for enhanced 3D perception and localisation accuracy. It also encompasses a YOLOv4-tiny network for dynamic interference removal – filtering out moving objects like workers and machinery as mentioned in Wang, 2024 [3]. In this setup, each new image frame is fed into the network, which uses deep CNNs to predict the bounding boxes and object categories, helping to identify and remove dynamic features from the frame, preventing inference with the mapping process. This technique resulted in a 19.08% reduction in path planning time and a 63.81% reduction in travel time meaning higher efficiency and accuracy in 3D map construction [3]. This technique is an example of how deep learning can be used to optimise existing algorithms so that MSINPS can deal with increasing amounts of input sensory data. Additionally, it must be noted that while optimised VSLAM could be seen as a cost-effective alternative to LiDAR-based methods, replacing LiDAR exclusively with cameras could result in safety issues, particularly in fully self-driving-enabled vehicles [8].

1.4 Deep Reinforcement Learning (DRL)

Deep Reinforcement Learning is a subfield of machine learning that combines deep learning and reinforcement learning (RL) to enable agents to learn optimal behaviours in complex environments through trial and error. There are several DRL algorithms; however, to address the high-dimensional continuous action space that is often the case for autonomous navigation certain algorithms perform better than others. One such algorithm, as suggested by Xu, L. et al., 2024 [2], is the Deep Deterministic Policy Gradient (DDPG) algorithm, which was designed for continuous action spaces. Xu, L. et al., 2024 [2] obtained results where DDPG performed the best with a 97% success rate against the more traditional DDQN (81%) and DQN (76%) algorithms using a four-wheeled vehicle that was trained to plan paths, avoid collisions and explore an unfamiliar setting consisting of a house and trees effectively. Proximal Policy Optimisation (PPO) [5] is one of many other algorithms suitable for navigation. DRL algorithms can also be combined with Learning from Demonstrations (LfD) methods like Behavioural Cloning (BC) and Inverse Reinforcement Learning (IRL) that allow the agent to learn a task by observing expert demonstrations that are typically in the form of state-action pairs. From the information obtained, it can be understood that DRL methods are highly versatile due to the range of available algorithms that can be utilised with complementary techniques like behavioural cloning to boost performance for a specific autonomous robot navigation task.

1.5 Conclusion

In conclusion, deep learning has had a largely positive impact on the autonomous navigation of robots. There do exist general drawbacks, such as the black-box nature of neural networks, making it difficult to understand deep learning decisions. High computational demand and unpredictable errors concerning physics simulation are also problems that are being addressed by building more efficient models and physics-constrained neural networks, respectively [4]. Finally, it can also be noted that CNN-based deep learning techniques have more to do with robot perception while DRL techniques are usually for learning navigation policies.

2 Methodology: Deep Learning for Vision-Based Recognition

2.1 Problem Definition

The objective of this study is to develop a deep learning model for object classification in robotics vision tasks. The model is trained using the CIFAR-100 dataset, which contains 100 object classes with 600 images per class. Each image is of size 32×32 pixels with three colour channels (RGB). Given the complexity of multi-class classification, a baseline Convolutional Neural Network (CNN) is designed to extract meaningful features and classify images accurately.

2.2 Baseline Network Architecture

The CNN model consists of multiple convolutional layers with increasing filter sizes, max-pooling layers for down-sampling, and fully connected (dense) layers for classification. The architecture is structured as follows:

- **Convolutional Layers:** The network includes 3 convolutional blocks, where the block filter sizes increase from 128,256 to 512 filters to encode more complex features as depth increases. Each block applies a 3x3 kernel and padding='same' (only in first layer of block), ensuring that height & width remain unchanged after the convolution operation. This is followed by a Rectified Linear Unit (ReLU) activation function to introduce non-linearity and improve learning. Multiple convolutional layers allow the network to capture complex features at various levels of abstraction - essential for image classification, as demonstrated in studies like [6].
- **Pooling Layers:** After every two convolutional layers (starting from layer 2 out of 6), a 2x2 max-pooling layer halves the spatial dimensions of the input each time if the stride is the default (which is the pool size), thereby decreasing computational load and overfitting. Max-pooling reduces dimensionality while retaining the most prominent features of the inputs. This approach is commonly employed in CNN architectures for image classification tasks [7].
- **Dropout Layers:** Dropout is implemented to randomly deactivate a fraction (0.25 to 0.5) of neurons during training - enhances model generalisation by preventing co-adaptation of feature detectors. [6].
- **Fully Connected (Dense) Layers:** The feature maps are flattened and passed through two dense layers, the first with 1024 neurons along with ELU activation, which allows small negative values unlike RELU to prevent dying neurons, and accelerate convergence, and the second with a softmax activation function corresponding to the number of classes to capture complex patterns and perform multi-class classification effectively. [7].

2.3 Baseline Experimental Setup

- **Dataset:** CIFAR-100, consisting of 60,000 images (50,000 for training, 10,000 for testing).
- **Preprocessing:** Images are normalised to the range [0,1] by dividing pixel values by 255. Gold Standard (output data), initially an integer label (0-99) is converted into a one-hot vector of length 100.
- **Training-Validation Split:** The training data is split in a standard manner into 80% training and 20% validation.
- **Optimiser and Loss:** Adam optimiser with a learning rate of 0.0001 and weight decay of $1e^{-6}$. Categorical Cross-Entropy Loss is used since outputs are one-hot vectors. Encourages the model to assign high probability to the correct class while minimising incorrect class probabilities.
- **Data Augmentation:** Horizontal flipping, width and height shifting (10%) are applied to enhance generalisation since these are distortions that frequently occur in reality [7]. Vertical flipping, on the other hand, where the images are flipped upside down, is not so common.
- **Hardware:** Kaggle Notebook with 2xT4 GPUs for acceleration.

2.4 Results and Analysis

In order to generate data for analysis, three model hyper-parameters were each varied three times, including their base values defined above. For each configuration of a particular hyperparameter, the model was trained over three independent runs, and the results were averaged. During each of these experiments, the model was trained for 20 epochs to get a meaningful idea of the trends and quality of the model when a particular hyper-parameter was changed. Note that all the data below has been rounded to 3 decimal places for readability.

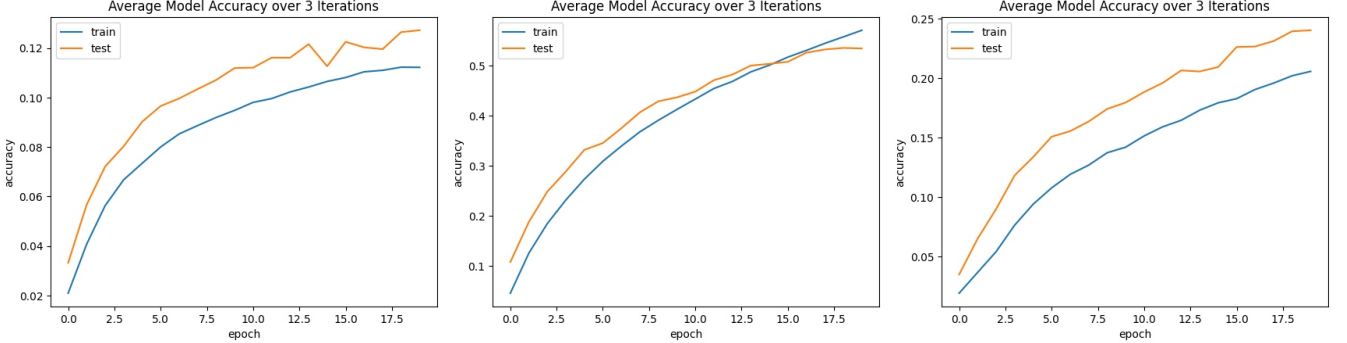


Figure 1: Average model accuracy over three runs for different learning rates (1×10^{-3} , 1×10^{-4} , 1×10^{-5} from left to right), showing train (blue) and test (orange) performance over 20 epochs.

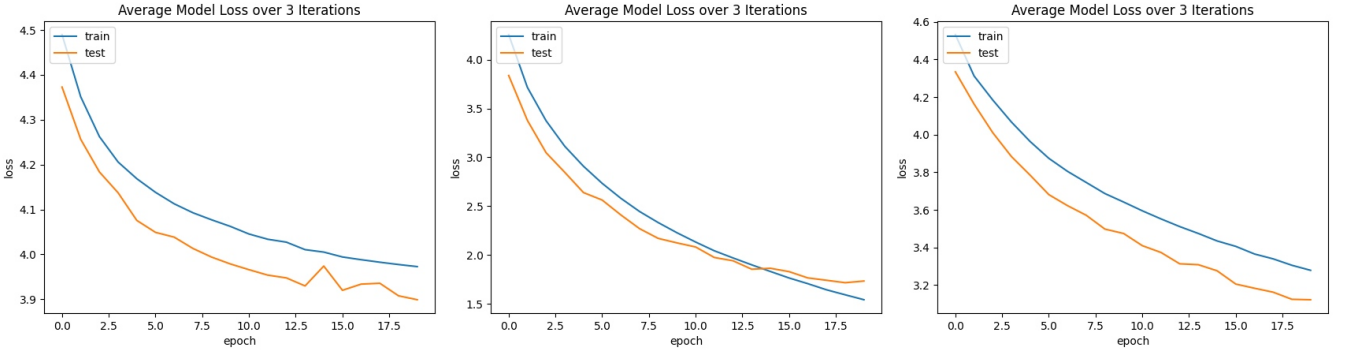


Figure 2: Average model loss over three runs for different learning rates (1×10^{-3} , 1×10^{-4} , 1×10^{-5} from left to right), showing train (blue) and test (orange) performance over 20 epochs.

Learning Rate	Run 1	Run 2	Run 3	Average
1×10^{-3}	0.010	0.365	0.010	0.128
1×10^{-4}	0.543	0.531	0.541	0.538
1×10^{-5}	0.248	0.252	0.230	0.243

Table 1: Test Accuracies and Averages for Different Learning Rates

As the learning rate changes in Figure 1, it can be seen that in all the learning rate graphs, there is a gradual increase in average accuracy over the epochs with diminishing returns as training/testing progresses (a logarithmic increase in average accuracy). The test average accuracies are also larger than the training accuracies for the learning rates 1×10^{-3} and 1×10^{-5} in all 20 epochs, while for a learning rate of 1×10^{-4} the test average accuracy is overtaken by the training accuracy at roughly the 14th epoch. The graphs representing the average loss over 20 epochs in Figure 2 are just a vertical flip of the graphs from Figure 1, meaning that they show logarithmic decay as opposed to increase, for each graph respectively, with a different range of (loss) values. This trend is observed in all the loss graphs for the other hyper-parameter configurations below, and as a result of this, the loss graphs henceforth are attached in the appendix since they provide almost identical insights to the average accuracy graphs.

With the 1×10^{-3} learning rate, the latter testing epochs in Figure 1 show signs of instability with the accuracy alternatively dipping sharply and then rising back above again. Table 1 also supports the notion of instability since run 1 and run 3 values (both 0.010) for the final testing results are much lower than the run 2 (0.365) value, giving a significant standard deviation of 16.7%. This inconsistency is a sign that the learning rate is too large, resulting in large step sizes, making it difficult for the Adam optimiser to aid in improving learning and generalisation of the model, and producing the worst final average testing accuracy results across the different learning rates.

A learning rate of 1×10^{-5} gives better results, with an average final testing accuracy of 0.243 and not as much variation across the runs (std dev = 0.94%), suggesting more stable learning. However it has quite a slow convergence

rate compared to the 1×10^{-4} which reaches a final testing accuracy of 0.538 - also with not much variation across runs (std dev = 0.55%) and a smooth curve in Figure 1, which indicates stability. It must be noted, though, that the test average accuracy is overtaken by the training accuracy as mentioned earlier and begins to plateau, suggesting that if the model was trained further, it would not generalise as well as over the first 20 epochs and not improve as much either.

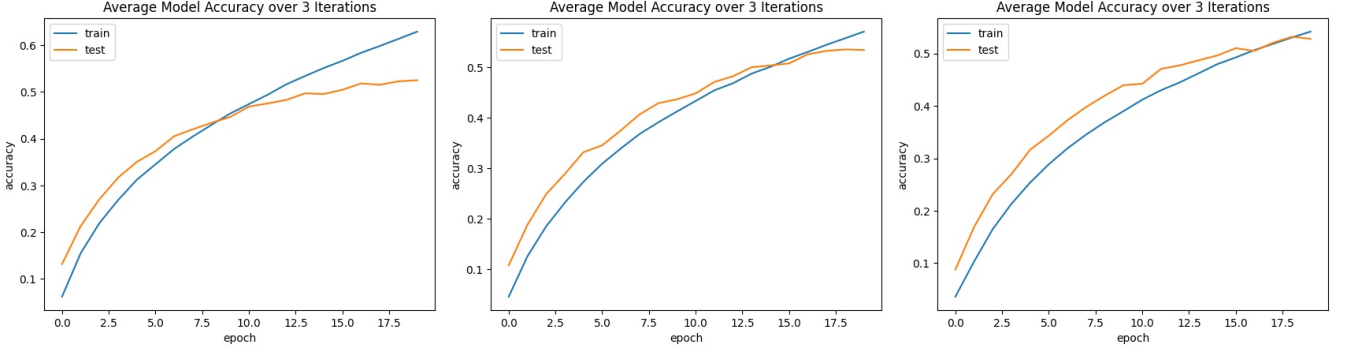


Figure 3: Average model accuracy over three runs for different dropout configurations where tuple represents dropout values from top to bottom-most layers ((0.1, 0.1, 0.3), (0.25, 0.25, 0.5), (0.3, 0.3, 0.6) from left to right), showing train (blue) and test (orange) performance over 20 epochs.

Dropouts	Run 1	Run 2	Run 3	Average
(0.1, 0.1, 0.3)	0.539	0.529	0.520	0.529
(0.25, 0.25, 0.5)	0.543	0.531	0.541	0.538
(0.3, 0.3, 0.6)	0.535	0.539	0.531	0.535

Table 2: Test Accuracies and Averages for Different Dropout Configurations

From Table 2, it can be observed that dropout results are fairly consistent across runs 1 to 3 for all of the dropout configurations. The final average testing results are also quite similar, with only a range of 0.009 between the best (0.25, 0.25, 0.5) and worst performing (0.1, 0.1, 0.3) configurations. This makes it seem like changing the dropout rate does not largely affect the model, however, Figure 3 gives a more comprehensive idea of what occurs. The lowest dropout configuration graph appears to show signs of over-fitting relatively early on since the average testing accuracy starts to increase at a lower rate than the training accuracy, causing it to be overtaken by the training accuracy at about the 8th epoch. When dropout is low, the model can memorise training data by heavily relying on certain neurons. This reduces generalisation to unseen data. For example, in the context of CIFAR100, a model trained on brightly lit apples in training might struggle with darker apples in test images because it learned background cues instead of object features. As the dropout configurations become higher, it is evident from the graphs that over-fitting (by the average test becoming lower than the training accuracy) occurs at later epochs - 14th for the middle and around 17th for the highest configuration. The fact that the highest dropout configuration has a slightly lower final test average accuracy of 0.535 as opposed to the middle configuration (0.538) suggests that it has begun underfitting, where too few active neurons make learning challenging, while the middle configuration appears to have the optimal number of active neurons in this case.

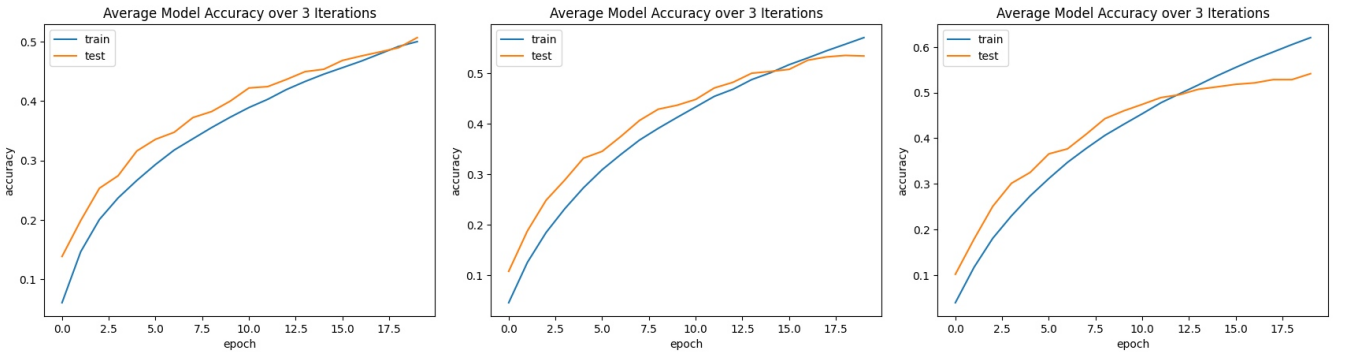


Figure 4: Average model accuracy over three runs for different kernel size configurations (All 6 of the convolutional layers with kernel size 2, 3, and the Top 4 layers having kernel size 3 while the bottom 2 layers have kernel size 5 from left to right), showing train (blue) and test (orange) performance over 20 epochs.

Kernel Configuration	Run1	Run 2	Run 3	Average
All 6 of the convolutional layers: 2	0.519	0.520	0.505	0.515
All 6 of the convolutional layers: 3	0.543	0.531	0.541	0.538
Top 4 layers: 3, Bottom 2 layers: 5	0.541	0.550	0.550	0.547

Table 3: Test Accuracies and Averages for Different Kernel Size Configurations across the 6 2D convolutional layers

Changing kernel size configuration, similar to dropout, produces somewhat consistent results across the test accuracy runs as seen in Table 3 - kernel size 2 gave the highest standard deviation but was only 0.68%. The final testing accuracy averages are also somewhat similar but not to the same degree as dropout (the range between greatest and least average is 0.032). Additionally, it can be inferred from Figure 4 that there is an over/under-fitting trend as seen with the dropout configuration, however the trend is reversed here with lower kernel size (all 6 layers size 2) appearing to be under-fitting to the training data relative the middle kernel size configuration (all 6 layers size 3), while the higher kernel size (mixed kernels size 3 and 5) over-fits. Smaller kernel sizes capture fine-grained details and reduce the risk of over-fitting but may miss larger patterns, requiring deeper networks for broader feature extraction. Larger kernels can capture global features more efficiently, but increase the risk of over-fitting due to more parameters. It must be noted that even though the graph suggests over-fitting, the higher kernel size final average test accuracy of 0.547 is the greatest out of all the configurations. Using smaller kernels on top and larger ones below allows the network to capture local details first, then combine them into global patterns, balancing fine-grained features and broader context and appears to perform best across 20 epochs.

2.5 Conclusion

The simulations revealed that a learning rate of 1×10^{-4} provided the most stable performance, while higher learning rates (1×10^{-3}) led to instability. Dropout configurations showed that a moderate dropout rate (0.25, 0.25, 0.5) struck the best balance between over-/under-fitting. Kernel size analysis indicated that a mix of smaller kernels on top and larger ones below offered the best performance, though very large kernels could risk over-fitting. The highest final average test accuracy of 0.547 after 20 epochs suggests that additional training could yield better results, as the model had not yet fully converged (plateaued). Future work could explore longer training durations (for instance would be interesting to see if the mixed kernel configuration improves beyond 0.547 or if it over-fits), different regularisation techniques (such as L2 regularisation or batch normalisation), and advanced model architectures like **ResNet** (which uses residual connections to allow deeper networks) or **DenseNet** (which connects each layer to every other layer, improving feature reuse), to further improve model performance and generalisation.

References

- [1] J. Priscilla Sasi et al. (2023) ‘Deep Learning Techniques for Autonomous Navigation of Underwater Robots’, 2018 5th IEEE Uttar Pradesh Section International Conference on Electrical, Electronics and Computer Engineering (UPCON), pp. 1630–1635. Available at: <https://doi.org/10.1109/upcon59197.2023.10434865>.
- [2] Xu, L. et al. (2024) ‘Autonomous Navigation of Unmanned Vehicle Through Deep Reinforcement Learning’, pp. 480–484. Available at: <https://doi.org/10.1145/3716895.3716980>.
- [3] Wang, X. (2024) ‘Autonomous Navigation of Construction Robots Based on Visual SLAM Technology’, pp. 139–143. Available at: <https://doi.org/10.1145/3653644.3658511>.
- [4] Fekry, A. et al. (2024) ‘Deep Learning-Based Strategies for Integrated Autonomous Navigation: A Review’, 2024 International Telecommunications Conference (ITC-Egypt), pp. 692–697. Available at: <https://doi.org/10.1109/itc-egypt61547.2024.10620533>.
- [5] Ravi, K.B. et al. (2020) Deep Reinforcement Learning for Autonomous Driving: A Survey, arXiv.org. Available at: <https://arxiv.org/abs/2002.00444>
- [6] Jost Tobias Springenberg, Alexey Dosovitskiy, Thomas Brox, Martin Riedmiller. ‘Striving for Simplicity: The All Convolutional Net.’ arXiv preprint arXiv:1412.6806 (2014). Available at: <https://arxiv.org/abs/1412.6806>.
- [7] Jason Brownlee. ‘How to Develop a CNN From Scratch for CIFAR-10 Photo Classification.’ Machine Learning Mastery, 2018. Available at: <https://machinelearningmastery.com/how-to-develop-a-cnn-from-scratch-for-cifar-10-photo-classification/>.
- [8] Altchek, A. (2025) YouTuber tests Tesla Autopilot against LiDAR car, Business Insider. Available at: <https://www.businessinsider.com/tesla-autopilot-vs-lidar-test-youtube-test-mark-rober-video-2025-3#:~:text=YouTuber%20Mark%20Rober%20tested%20Tesla's,tests%2C%20unlike%20the%20lidar%20systems.>

3 Appendix

A Code Link

The code and experiments can be accessed at: [GitHub Repository Link](#).

B Loss figures for dropout and kernel size configurations

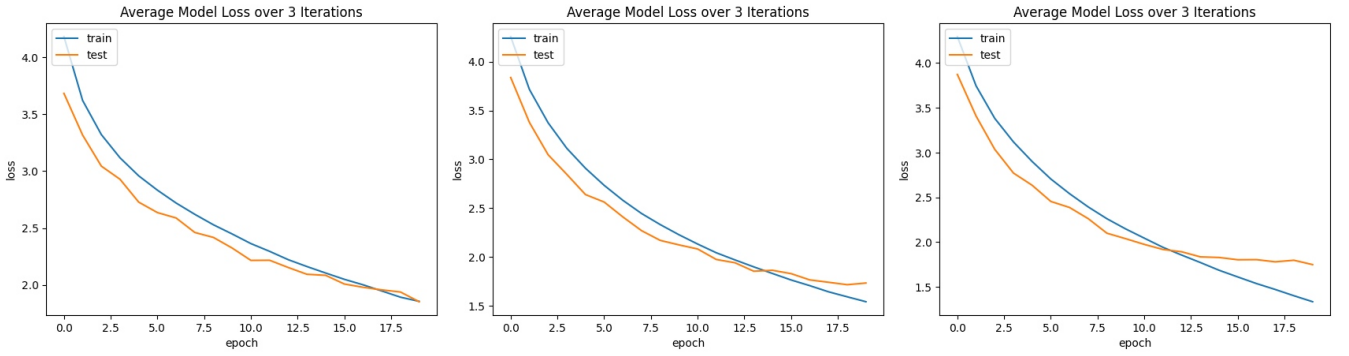


Figure 5: Average model loss over three runs for different kernel size configurations (All 6 of the convolutional layers with kernel size 2, 3, and the Top 4 layers having kernel size 3 while the bottom 2 layers have kernel size 5 from left to right), showing train (blue) and test (orange) performance over 20 epochs.

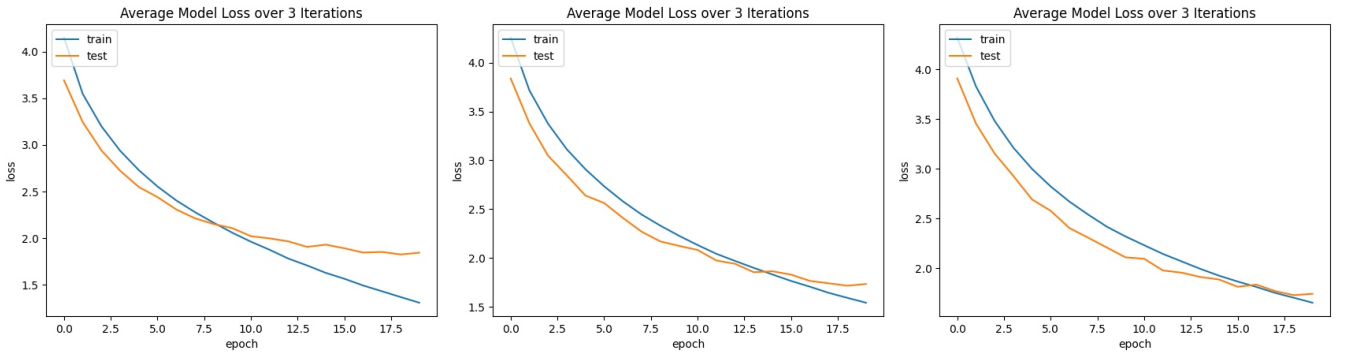


Figure 6: Average model loss over three runs for different dropout configurations where tuple represents dropout values from top to bottom-most layers ((0.1, 0.1, 0.3), (0.25, 0.25, 0.5), (0.3, 0.3, 0.6) from left to right), showing train (blue) and test (orange) performance over 20 epochs.

C Raw Data

Table 4: Validation Accuracy for Different Learning Rates Over 20 Epochs

Epoch	1e⁻³			1e⁻⁴			1e⁻⁵		
	Run 1	Run 2	Run 3	Run 1	Run 2	Run 3	Run 1	Run 2	Run 3
1	0.009	0.082	0.009	0.118	0.088	0.118	0.036	0.034	0.034
2	0.009	0.151	0.010	0.189	0.184	0.190	0.061	0.069	0.065
3	0.009	0.198	0.009	0.255	0.238	0.254	0.087	0.092	0.092
4	0.009	0.223	0.009	0.302	0.288	0.277	0.116	0.118	0.121
5	0.009	0.252	0.010	0.339	0.335	0.322	0.137	0.133	0.132
6	0.009	0.270	0.011	0.356	0.332	0.349	0.151	0.150	0.152
7	0.012	0.278	0.009	0.377	0.359	0.389	0.151	0.160	0.156
8	0.009	0.291	0.010	0.415	0.402	0.403	0.163	0.164	0.163
9	0.010	0.303	0.009	0.426	0.434	0.426	0.176	0.173	0.173
10	0.010	0.317	0.009	0.444	0.431	0.435	0.181	0.189	0.169
11	0.009	0.318	0.009	0.451	0.447	0.447	0.190	0.191	0.185
12	0.009	0.329	0.011	0.476	0.456	0.481	0.197	0.197	0.194
13	0.008	0.331	0.010	0.485	0.478	0.484	0.205	0.211	0.203
14	0.010	0.346	0.009	0.504	0.486	0.510	0.204	0.211	0.203
15	0.010	0.318	0.010	0.524	0.487	0.499	0.200	0.219	0.209
16	0.010	0.349	0.009	0.505	0.505	0.513	0.227	0.230	0.222
17	0.009	0.341	0.011	0.529	0.522	0.527	0.228	0.233	0.220
18	0.009	0.341	0.009	0.531	0.532	0.534	0.230	0.234	0.231
19	0.009	0.361	0.010	0.536	0.529	0.541	0.238	0.250	0.230
20	0.009	0.364	0.009	0.534	0.531	0.538	0.244	0.250	0.228

Table 5: Validation Accuracy for Different Dropout Configurations Over 20 Epochs

Epoch	(0.1, 0.1, 0.3)			(0.25, 0.25, 0.5)			(0.3, 0.3, 0.6)		
	Run 1	Run 2	Run 3	Run 1	Run 2	Run 3	Run 1	Run 2	Run 3
1	0.129	0.131	0.135	0.118	0.087	0.118	0.086	0.095	0.082
2	0.214	0.205	0.221	0.189	0.184	0.190	0.165	0.167	0.175
3	0.277	0.270	0.264	0.255	0.238	0.254	0.232	0.231	0.232
4	0.314	0.315	0.320	0.302	0.287	0.277	0.272	0.264	0.275
5	0.355	0.343	0.353	0.339	0.334	0.322	0.325	0.309	0.317
6	0.376	0.377	0.367	0.356	0.332	0.349	0.345	0.339	0.347
7	0.403	0.410	0.403	0.377	0.359	0.389	0.369	0.373	0.376
8	0.425	0.418	0.417	0.415	0.402	0.403	0.395	0.398	0.402
9	0.436	0.435	0.431	0.426	0.434	0.426	0.424	0.423	0.413
10	0.443	0.449	0.446	0.444	0.431	0.435	0.440	0.447	0.432
11	0.475	0.466	0.466	0.476	0.447	0.447	0.467	0.452	0.467
12	0.481	0.474	0.471	0.485	0.456	0.481	0.484	0.468	0.458
13	0.492	0.499	0.493	0.504	0.478	0.486	0.493	0.497	0.484
14	0.498	0.496	0.492	0.524	0.486	0.510	0.501	0.497	0.492
15	0.510	0.498	0.505	0.505	0.487	0.499	0.507	0.501	0.515
16	0.515	0.520	0.520	0.529	0.505	0.513	0.509	0.522	0.516
17	0.517	0.519	0.510	0.531	0.522	0.528	0.523	0.529	0.536
18	0.525	0.515	0.528	0.536	0.532	0.534	0.522	0.522	0.528
19	0.536	0.520	0.513	0.533	0.529	0.541	0.521	0.511	0.540
20	0.536	0.529	0.510	0.534	0.531	0.538	0.533	0.531	0.520

Table 6: Validation Accuracy for Different Kernel Size Configurations Over 20 Epochs

Epoch	All 6 of the conv layers: 2			All 6 of the conv layers: 3			Top 4 layers: 3, Bottom 2 layers: 5		
	Run 1	Run 2	Run 3	Run 1	Run 2	Run 3	Run 1	Run 2	Run 3
1	0.140	0.137	0.139	0.118	0.088	0.118	0.099	0.094	0.112
2	0.212	0.184	0.201	0.189	0.184	0.190	0.167	0.185	0.185
3	0.257	0.256	0.248	0.255	0.238	0.254	0.248	0.254	0.250
4	0.277	0.270	0.276	0.302	0.287	0.277	0.295	0.306	0.303
5	0.320	0.318	0.311	0.339	0.335	0.322	0.317	0.323	0.335
6	0.333	0.334	0.340	0.356	0.332	0.349	0.357	0.375	0.365
7	0.348	0.346	0.349	0.377	0.359	0.389	0.373	0.376	0.381
8	0.387	0.369	0.361	0.415	0.402	0.403	0.387	0.408	0.431
9	0.396	0.373	0.377	0.426	0.434	0.426	0.443	0.443	0.442
10	0.412	0.393	0.395	0.444	0.431	0.435	0.448	0.463	0.469
11	0.427	0.412	0.427	0.451	0.447	0.447	0.473	0.477	0.473
12	0.417	0.412	0.444	0.476	0.456	0.481	0.477	0.496	0.494
13	0.440	0.433	0.436	0.485	0.478	0.484	0.486	0.495	0.505
14	0.454	0.437	0.458	0.504	0.486	0.510	0.503	0.513	0.507
15	0.448	0.456	0.457	0.524	0.487	0.499	0.501	0.516	0.521
16	0.463	0.470	0.472	0.505	0.505	0.513	0.508	0.521	0.526
17	0.478	0.472	0.478	0.529	0.522	0.527	0.511	0.520	0.534
18	0.496	0.470	0.482	0.531	0.532	0.534	0.527	0.524	0.535
19	0.489	0.478	0.503	0.536	0.529	0.541	0.534	0.526	0.525
20	0.510	0.513	0.498	0.534	0.531	0.538	0.537	0.543	0.544